

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1511

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 40

Duration :60 Minutes

Code of Conduct

I **NIVASINI VM 192511356** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Nivasini VM**

Assessment Tasks

Industry Problem Solving Task

1. **Retail Data Optimization Problem** - A retail chain operates both physical stores and an online platform. It generates large volumes of sales, inventory, and customer behavior data. The company wants real-time inventory tracking and demand forecasting. It also aims to deliver personalized marketing recommendations to customers. Design a big data architecture to support these requirements. Explain the tools, technologies, and analytics models you would use.

Retail Data Optimization Using Big Data Architecture

Introduction

A modern retail chain generates huge amounts of data from physical stores, online shopping platforms, inventory systems, customer transactions, mobile applications, websites, and social media. To manage this data efficiently, the company can implement a Big Data Architecture that supports real-time inventory tracking, demand forecasting, and personalized marketing recommendations.

Data Sources

The system collects data from:

- Point-of-Sale (POS) systems in physical stores.
- Online shopping websites and mobile applications.
- Inventory and warehouse management systems.
- Customer purchase history.
- Product searches and browsing behavior.
- Customer reviews and feedback.
- Promotional and seasonal sales data.
- Supplier and delivery information.

Data Ingestion

Apache Kafka can be used to collect and stream large volumes of data in real time. For example, when a product is purchased in a store, the transaction can immediately be sent to Kafka.

Other tools such as Apache Flume can be used for collecting log-based data.

This layer ensures that data from different sources reaches the processing system continuously and reliably.

Data Storage

A Data Lake using technologies such as HDFS or cloud object storage can store large amounts of structured, semi-structured, and unstructured data.

- HDFS/Amazon S3/Azure Data Lake – large-scale data storage.

- Hive/BigQuery – analytical data warehouse.
- MongoDB/Cassandra – fast storage and retrieval of semi-structured data.

Historical sales, customer behavior, inventory records, and product information can be retained for long-term analysis.

Demand Forecasting

Machine-learning models can predict future product demand using historical and real-time data.

Useful models include:

- **ARIMA** – time-series forecasting.
- **Random Forest** – demand prediction using multiple factors.
- **XGBoost** – accurate prediction for complex datasets.
- **LSTM** – forecasting demand patterns over time.

Factors such as previous sales, season, holidays, discounts, weather, location, and customer behavior can be used as input variables.

For example, if sales of umbrellas increase before rainy periods, the system can forecast higher demand and recommend increasing inventory.

Personalized Marketing Recommendations

A recommendation engine analyzes customer behavior and purchase history to provide personalized product suggestions.

Methods include:

- **Collaborative Filtering** – recommends products based on similar customers.
- **Content-Based Filtering** – recommends products based on product characteristics.
- **Hybrid Recommendation** – combines both approaches.

Data Processing

Apache Spark can process large datasets efficiently. Spark Streaming can process incoming transaction and inventory data in near real time.

Processing activities include:

- Removing duplicate records.
- Handling missing values.
- Combining data from different stores.
- Calculating sales and inventory levels.

- Identifying frequently purchased products.
- Preparing data for machine-learning models.

Analytics and Visualization

Tools such as Power BI, Tableau, or Grafana can display:

- Current inventory levels.
- Daily and monthly sales.
- Best-selling products.
- Store-wise performance.
- Forecasted demand.
- Customer purchasing patterns.
- Low-stock alerts.

Security and Governance

The architecture should also provide:

- Data encryption.
- User authentication and authorization.
- Role-based access control.
- Data backup and recovery.
- Data quality monitoring.
- Customer privacy protection.

Benefits

The proposed Big Data architecture provides:

1. Real-time inventory monitoring.
2. Accurate demand forecasting.
3. Reduced stock-outs and overstock.
4. Better supply-chain management.
5. Personalized customer recommendations.

Conclusion

A Big Data architecture using Kafka, Spark, HDFS/cloud storage, NoSQL databases, machine-learning models, and visualization tools can effectively manage the retail chain's large-scale data. Real-time processing enables continuous inventory tracking, while predictive analytics helps forecast demand. Recommendation algorithms further improve customer engagement by providing personalized offers and products. Thus, the architecture helps the retailer reduce costs, improve inventory efficiency, and increase customer satisfaction.

2. **Social Media Fraud Detection Problem** - A social media platform processes billions of user interactions daily. These include posts, comments, likes, and shares. The platform wants to detect fake accounts and misinformation quickly. Real-time or near real-time processing is required. Design a scalable big data solution for this use case. Explain the role of machine learning and stream processing frameworks.

Social Media Fraud Detection Problem – Big Data Solution

Introduction

Social media platforms generate billions of user interactions every day in the form of posts, comments, likes, shares, messages, follows, and account activities. Due to this huge volume of data, detecting fake accounts, bots, spam, fraud, and misinformation is a major challenge. Fraudulent accounts can spread false information, create fake engagement, manipulate trends, and affect genuine users. Therefore, the platform requires a scalable Big Data solution that can process large amounts of data in real time or near real time.

Big Data Architecture

The proposed system begins by collecting data from different sources such as user posts, comments, likes, shares, login activities, account information, follower relationships, device information, IP addresses, and user reports. Since this data is generated continuously, a distributed data ingestion system is required.

After data ingestion, the events are sent to a stream processing framework such as Apache Flink or Apache Spark Streaming. These frameworks process incoming data continuously instead of waiting for all the data to be collected. This is important because fraudulent activity needs to be detected quickly.

Role of Stream Processing

Stream processing is responsible for analyzing user activities as they occur. For example, if a newly created account starts posting hundreds of comments within a few minutes, the system can immediately identify this as unusual behavior.

The stream processing layer performs operations such as data cleaning, filtering, aggregation, feature extraction, and detection of unusual patterns. It can calculate the number of posts made by a user, frequency of likes and shares, repeated content, sudden changes in activity, and other behavioral characteristics.

Machine Learning for Fake Account Detection

Machine learning plays an important role in identifying fake accounts and automated bots. Instead of manually checking every account, machine-learning models can learn patterns from previously identified genuine and fraudulent accounts.

Important features used for fake account detection include account age, number of followers, number of following accounts, posting frequency, number of likes and comments, repeated posts, profile information, login behavior, device information, and relationships with other suspicious accounts.

Models such as Random Forest, Logistic Regression, XGBoost, and Neural Networks can be used for classification. The model can produce a fraud probability or risk score for each account. Accounts with high risk scores can be automatically flagged for further investigation.

For example, an account created recently that follows thousands of users, posts identical content repeatedly, and performs hundreds of actions within a short period may receive a high fraud score.

Misinformation Detection

The system should also detect potentially misleading or false information. Natural Language Processing can be used to analyze the text of posts and comments.

NLP techniques can identify suspicious keywords, repeated claims, unusual language patterns, and similarities between different posts. Machine-learning models can classify content into categories such as normal, suspicious, or potentially misleading.

The system can also analyze shared URLs, sources, previous fact-checking results, and the speed at which a particular claim is being shared. Posts with a high misinformation risk can be sent to a human moderation team or an additional verification system.

It is better to use automated detection as a decision-support mechanism rather than automatically removing every post identified as suspicious, because machine-learning models can sometimes produce incorrect results.

Data Storage

The large amount of historical social media data can be stored using distributed storage technologies such as HDFS or cloud-based data lakes. These systems provide scalable storage for structured and unstructured data.

NoSQL databases such as Cassandra or MongoDB can be used when fast access to user and activity information is required. Historical data is important because it can be used for analyzing long-term behavior and training machine-learning models.

The stored data can also be used to identify new fraud patterns and improve the accuracy of the detection system over time.

Real-Time Detection Process

When a user performs an activity, the event is immediately sent to Apache Kafka. Kafka distributes the event to the stream-processing system. Apache Flink or Spark Streaming then processes the event and extracts useful features.

The machine-learning model analyzes these features and generates a risk score. If the activity is normal, it can continue without interruption. If the activity appears suspicious, the system can generate an alert, temporarily restrict the account, or send the case to human moderators.

For example, if hundreds of accounts suddenly share the same message within a few seconds, the system can detect the unusual pattern and investigate whether the accounts are part of a coordinated fake-account network.

Scalability

Since the platform handles billions of interactions, the architecture must be scalable. Kafka supports distributed data ingestion through partitions and multiple brokers. Flink and Spark can distribute processing across multiple nodes.

The system should also use load balancing and fault-tolerant mechanisms so that the failure of one server does not stop the entire fraud detection system.

Monitoring and Alerts

A monitoring system can be used to track the performance of the fraud detection platform. It can display the number of suspicious accounts detected, number of potentially misleading posts, processing speed, detection accuracy, and system latency.

Advantages

The proposed Big Data solution provides several advantages. It can process huge volumes of social media data efficiently and detect suspicious activities with low latency. Machine learning reduces the need for manual checking and can identify complex patterns that may not be easily detected using simple rules.

Real-time stream processing helps prevent fraudulent activities from spreading widely. Historical data storage allows models to be continuously trained and improved. The scalable architecture also allows the system to handle increasing numbers of users and interactions.

Conclusion

A combination of Apache Kafka, Apache Flink or Spark Streaming, distributed storage, machine learning, and NLP provides an effective solution for social media fraud detection. Kafka handles the continuous flow of user interactions, while stream-processing frameworks analyze the data in real time or near real time. Machine-learning models detect fake accounts, bots, abnormal behavior, and potentially misleading content. Distributed storage maintains historical information for analysis and model training.

Such a scalable Big Data architecture enables social media platforms to detect threats quickly, reduce fraudulent activities, improve content moderation, and provide a safer and more trustworthy environment for users.

3. Government Data Platform Problem A government agency is building a national data platform.

It combines census, economic, and geospatial datasets. Different departments need access to shared and integrated data. The system must ensure interoperability across multiple agencies. Strict privacy and security requirements must be followed. Propose a governance and data management strategy.

Government Data Platform – Governance and Data Management Strategy

Introduction

A government agency developing a national data platform needs to combine large amounts of data from different sources such as census records, economic information, land records, transportation data, and geospatial datasets. Since different government departments need to access and share this information, the platform must provide proper data integration and interoperability. At the same time, government data may contain sensitive personal and confidential information, so strong privacy, security, and governance mechanisms are required.

Data Governance

Data governance defines how government data is collected, stored, shared, accessed, and managed. A central data governance committee can establish common policies and standards for all participating departments.

The governance framework should define rules for data collection, storage, sharing, retention, modification, and deletion. It should also establish standards for handling sensitive and confidential information.

Data Integration

The platform should integrate data from different government departments into a common environment. Census data, economic data, and geospatial information may have different formats and structures, so the data must be cleaned and transformed before integration.

For real-time or frequently updated information, APIs and event-based data integration can be used. This allows departments to exchange information without maintaining separate copies of the same dataset.

Interoperability

Interoperability is one of the most important requirements of a national data platform. Different departments may use different databases, applications, and data formats. Therefore, common data standards should be established.

A centralized metadata catalogue can describe what each dataset contains, where it came from, how frequently it is updated, and which department owns it. This makes datasets easier to discover and understand.

Data Quality Management

Government decisions depend heavily on accurate data. Therefore, data quality must be continuously monitored.

The system should check for:

- Missing values.
- Duplicate records.
- Incorrect formats.
- Inconsistent information.
- Outdated records.
- Invalid geographical information.

Data Security

Strong security controls are required because government platforms may contain sensitive citizen and national information.

The system should use encryption for data both during transmission and while stored. Secure communication protocols should be used when information is exchanged between departments.

Privacy Protection

Privacy protection is essential when handling census and citizen-related information. Personal information should only be collected and used for legitimate government purposes.

The platform should follow applicable data protection and privacy laws. Data retention policies should also specify how long different types of information can be stored.

Access Management and Data Sharing

Different departments should have controlled access to shared datasets. A centralized identity and access management system can authenticate users and applications.

Data can be classified into categories such as public, internal, confidential, and highly sensitive. A national data catalogue should be created to provide information about available datasets. It should contain the dataset name, description, owner, source, format, update frequency, sensitivity level, and access restrictions.

For example, a department searching for population information should be able to find the appropriate census dataset and understand its contents before requesting access.

Metadata also improves interoperability because different departments can understand the meaning and structure of shared data.

Geospatial Data Management

Since the platform includes geospatial datasets, common geographic standards and coordinate systems should be used. Roads, districts, states, land boundaries, and other geographic information should follow standardized formats.

Geospatial data should be integrated with census and economic datasets so that departments can perform location-based analysis. For example, the government could analyze population, employment, infrastructure, and economic conditions at the district or regional level.

Data Lifecycle Management

Government data should be managed throughout its complete lifecycle. The lifecycle includes data creation, collection, storage, processing, sharing, archival, and deletion.

Old or unused data should be archived according to government policies. Data that is no longer required should be securely deleted when legally permitted.

A proper lifecycle strategy reduces storage costs and prevents unnecessary retention of sensitive information.

Auditing and Monitoring

The system should monitor:

- Unauthorized access attempts.
- Unusual data downloads.
- Changes to important datasets.
- Failed authentication attempts.
- Data-sharing activities.
- Security incidents.

Conclusion

A national government data platform should be supported by a strong data governance and management framework. Standardized data formats, metadata, APIs, data-quality controls, and integration processes ensure interoperability between departments. At the same time, encryption, access control, authentication, privacy protection, auditing, and data classification provide the required security.

By implementing these measures, the government can create a secure and reliable national data platform that enables departments to share information effectively while protecting citizen privacy and supporting better policy-making and public services.

4. **Banking Fraud Detection Problem**

A bank processes millions of financial transactions every minute. It needs to detect fraudulent activities in real time. The system must analyze historical and streaming transaction data. Low latency and high accuracy are critical requirements. Design a big data pipeline for fraud detection. Explain the technologies and algorithms you would implement.

Banking Fraud Detection Using Big Data

Introduction

Banks process millions of transactions every minute through ATMs, credit cards, debit cards, mobile banking, internet banking, and online payment systems. With such a large volume of transactions, detecting fraudulent activities manually is difficult. Fraud must be identified quickly because even a few seconds of delay can result in significant financial losses.

A Big Data fraud detection system should combine historical data, real-time streaming data, machine learning, distributed processing, and secure storage. The main objectives are low latency, high accuracy, scalability, and secure processing.

Big Data Pipeline

The system first collects transaction data from different sources such as ATM transactions, card payments, online banking, mobile applications, and payment gateways. Each transaction may contain information such as transaction amount, location, time, device, merchant, account details, and transaction history.

Apache Kafka can be used for real-time data ingestion. Every transaction is published as an event to Kafka. Kafka supports high-throughput and distributed processing, making it suitable for millions of transactions.

The streaming data can then be processed using Apache Flink or Apache Spark Streaming. These frameworks analyze transactions continuously and calculate important features such as transaction frequency, spending patterns, location changes, and unusual transaction amounts.

Data Storage

Historical transaction data can be stored in a distributed data lake using technologies such as HDFS or cloud storage. This historical information is useful for identifying long-term customer behavior and training machine-learning models.

A database such as Cassandra or another NoSQL database can be used when very fast access to recent transaction information is required. A data warehouse can also be used for reporting and detailed historical analysis.

The system should maintain both historical and streaming data because fraud detection requires comparison between the current transaction and previous customer behavior.

Real-Time Stream Processing

When a transaction occurs, it is immediately sent to Kafka. The stream-processing system receives the transaction and performs preprocessing and feature extraction.

For example, the system can calculate:

- Number of transactions made in the last few minutes.
- Average transaction amount of the customer.
- Difference between current and previous transaction locations.
- Number of failed transactions.
- Device and IP address changes.
- Unusual spending patterns.
- Frequency of transactions at a particular merchant.

These features are then passed to the fraud detection model.

Machine Learning Algorithms

Machine learning is used to identify patterns associated with fraudulent transactions. Both supervised and unsupervised learning techniques can be used.

Random Forest can be used as a classification algorithm to distinguish between genuine and fraudulent transactions. It works well with multiple transaction features and can provide good accuracy.

XGBoost can also be used because it performs effectively on structured transaction data and can identify complex relationships between features.

For detecting previously unknown fraud patterns, Isolation Forest can be used as an anomaly detection algorithm. It identifies transactions that are significantly different from normal customer behavior.

Other methods such as Logistic Regression and Neural Networks can also be used depending on the required accuracy and processing time.

Fraud Risk Scoring

Instead of simply classifying every transaction as fraud or genuine, the system can generate a fraud risk score.

For example:

- Low risk → Transaction is approved.
- Medium risk → Additional verification is requested.
- High risk → Transaction is blocked or sent for investigation.

This approach reduces false positives and prevents genuine customers from being unnecessarily blocked.

Low-Latency Processing

Low latency is extremely important in banking fraud detection. The system should analyze a transaction within a very short time before completing the payment.

Kafka provides fast event delivery, while Flink or Spark Streaming performs distributed processing. Machine-learning models should be optimized for fast inference so that a prediction can be generated quickly.

Frequently accessed information can also be stored in high-speed databases or caches to reduce response time.

Scalability and Fault Tolerance

The system should be capable of handling increasing transaction volumes. Kafka partitions allow transactions to be distributed across multiple servers. Flink or Spark can process these streams in parallel.

Cloud infrastructure can provide additional computing resources when transaction traffic increases. Replication and fault-tolerance mechanisms should be implemented so that failure of one server does not stop fraud detection.

Advantages

The proposed Big Data pipeline provides:

1. Real-time fraud detection.
2. Low-latency transaction processing.
3. High-volume transaction handling.
4. Improved fraud detection accuracy.
5. Detection of both known and unknown fraud patterns.
6. Continuous machine-learning improvement.
7. Scalable and fault-tolerant processing.
8. Secure handling of financial data.

Conclusion

A Big Data fraud detection pipeline using Apache Kafka, Apache Flink or Spark Streaming, distributed storage, NoSQL databases, and machine-learning algorithms can effectively detect banking fraud in real time. Historical data is used to train accurate models, while streaming data is analyzed immediately as transactions occur.

5. Agriculture Smart Farming Problem

A smart farming system collects soil, weather, and crop data. Sensors and drones provide real-time field information. Farmers want insights to improve yield and resource usage. Data is generated from multiple distributed sources. Design a big data solution for precision agriculture. Explain how analytics can support decision-making.

Agriculture Smart Farming – Big Data Solution for Precision Agriculture

Introduction

Smart farming uses modern technologies such as IoT sensors, drones, GPS, cloud computing, and Big Data analytics to improve agricultural production. A smart farming system continuously collects data about soil, weather, crops, water usage, and field conditions from multiple distributed sources. Since this data is generated in large volumes and at different locations, a scalable Big Data solution is required.

The main objective of precision agriculture is to provide farmers with accurate information about crop health, irrigation, fertilizers, pests, and weather conditions. This helps farmers increase crop yield while reducing the unnecessary use of water, fertilizers, pesticides, and other resources.

Data Sources

The system collects data from different sources. Soil sensors measure parameters such as soil moisture, temperature, pH, and nutrient levels. Weather stations provide information about temperature, rainfall, humidity, wind speed, and sunlight.

Drones equipped with cameras can capture aerial images of agricultural fields. These images can be used to identify crop diseases, damaged areas, weed growth, and variations in crop health. GPS devices provide accurate location information for different areas of the farm.

Other data sources include irrigation systems, satellite images, historical crop records, fertilizer usage, and previous yield information. Combining these sources provides a complete view of field conditions.

Data Collection and Ingestion

IoT sensors continuously generate data from different locations in the farm. A data ingestion system collects this information and sends it to a central platform.

Apache Kafka or similar messaging systems can be used to collect and transfer real-time sensor data. The system can handle data from thousands of sensors and distribute it to different processing services.

For drone and satellite images, cloud storage or a data lake can be used because these files can be very large. The collected data is then prepared for further processing and analysis.

Data Storage

A cloud-based data lake can store large volumes of agricultural data. Technologies such as HDFS, Amazon S3, or Azure Data Lake can be used for scalable storage.

Structured sensor data can be stored in databases, while large drone images and satellite data can be stored as files in the data lake. Historical data should also be maintained because it helps in identifying long-term patterns in crop growth, weather, and soil conditions.

Data Processing

The collected data needs to be cleaned and processed before analysis. Apache Spark can be used for large-scale processing of historical data. For real-time sensor information, Apache Spark Streaming or Apache Flink can process data continuously.

Data processing includes removing incorrect sensor readings, handling missing values, combining information from different sensors, and converting the data into useful features.

For example, soil moisture data can be combined with weather forecasts to determine whether a particular field requires irrigation.

Analytics for Crop Monitoring

Big Data analytics can help farmers continuously monitor crop conditions. Data from sensors, drones, and satellites can be analyzed to identify areas where crops are growing poorly.

Image-processing and machine-learning techniques can detect changes in crop color, leaf condition, plant density, and other visual characteristics. This can help identify diseases, pest attacks, water stress, and nutrient deficiencies at an early stage.

Farmers can then take action only in the affected areas instead of treating the entire field.

Irrigation Management

One of the major applications of precision agriculture is smart irrigation. Soil moisture sensors continuously monitor the amount of water present in the soil.

Machine-learning models can combine soil moisture, temperature, rainfall forecasts, crop type, and historical irrigation data to predict the water requirements of crops.

If the soil moisture falls below the required level, the system can generate an irrigation recommendation or automatically activate a smart irrigation system. This reduces water wastage and ensures that crops receive the required amount of water.

Crop Yield Prediction

Machine-learning algorithms can be used to predict crop yield before harvesting. Historical yield data can be combined with soil conditions, weather information, crop variety, fertilizer usage, and irrigation data.

Algorithms such as Random Forest, Decision Trees, Regression models, and XGBoost can be used for prediction. The predicted yield helps farmers plan harvesting, storage, transportation, and sales.

Weather and Risk Analysis

Weather conditions have a major effect on agriculture. The system can analyze current and historical weather information to identify possible risks such as drought, heavy rainfall, extreme temperatures, or strong winds.

Predictive analytics can provide early warnings to farmers. For example, if heavy rainfall is expected, farmers can delay irrigation or take steps to protect crops. Similarly, drought predictions can help farmers plan water resources more efficiently.

Pest and Disease Detection

Drone images and sensor data can be analyzed using machine learning and image-processing techniques to identify crop diseases and pest attacks.

Image classification models can identify abnormal patterns in plant leaves or crops. When a disease is detected, the system can identify the affected region and alert the farmer.

This supports targeted pesticide application, reducing chemical usage and protecting healthy parts of the field.

Decision-Making Support

The final analytics results can be presented through a mobile application or web dashboard. Farmers can receive information about soil conditions, irrigation requirements, crop health, weather risks, and expected yield.

For example, the system may recommend increasing irrigation in one section of a field while reducing irrigation in another section based on soil moisture levels. Similarly, fertilizer recommendations can be generated based on soil nutrient levels and crop requirements.

This converts raw sensor data into practical decisions that farmers can easily understand and use.

Benefits

The proposed Big Data solution provides several benefits:

1. Improves crop yield through data-driven farming.
2. Reduces water consumption through smart irrigation.
3. Reduces unnecessary fertilizer and pesticide usage.
4. Detects crop diseases at an early stage.
5. Provides accurate crop yield predictions.
6. Helps farmers respond to weather risks.

7. Supports real-time monitoring of agricultural fields.
8. Reduces operational costs and resource wastage.
9. Enables better planning and decision-making.
10. Supports sustainable and environmentally friendly agriculture.

Conclusion

A Big Data solution combining IoT sensors, drones, cloud storage, Apache Kafka, Apache Spark or Flink, machine learning, and analytics dashboards can provide an effective precision agriculture system. Real-time sensor data helps monitor soil and crop conditions, while drone and satellite data provide detailed information about crop health.

Machine-learning models can predict crop yield, irrigation requirements, diseases, pests, and weather-related risks. These analytics provide farmers with timely and accurate recommendations, allowing them to use water, fertilizers, pesticides, and other resources efficiently. Therefore, Big Data and analytics can improve agricultural productivity, reduce costs, and support sustainable smart farming.